Presentation script

CSE 705 · Cora GNN study · 20 minutes

1. Title slide

0:40

Good afternoon. My name is Kiarash Ara, and this is joint work with Yiheng Wu at McMaster University.

Our project studies what happens to graph neural networks when they get deeper. The short version is this. Depth does not fail in one single way. It fails differently depending on the architecture.

We measured that on the Cora citation network, across four architectures, five depths, and ten random seeds. Over the next twenty minutes I will show you how we separated those failure modes, and why accuracy alone cannot do it.

2. Title slide

1:00

Let me start with why depth matters at all.

A graph neural network passes messages between connected nodes. Each layer lets a node see one hop further into the graph. So depth is how the model widens its view.

For a graph convolutional network, one layer looks like the equation on the screen. We multiply by a normalized propagation operator, multiply by a weight matrix, and apply a nonlinearity.

The problem is that applying that operator again and again pulls node representations toward one another. Nodes that started out different become similar.

The panel on the left illustrates that on a small toy graph. It is only an illustration. Every claim I make after this slide comes from measurements on Cora. The question we asked is whether that contraction looks the same for every architecture.

3. The attribution problem — accuracy cannot name the mechanism

1:05

This slide is the reason the project is designed the way it is.

Suppose a deep model performs badly. There are two very different explanations.

The first is that the model never trained. The optimizer never moved the weights. Whatever the representation looks like, it is what random propagation produced, not what learning produced.

The second is that the model did train, and then repeated propagation contracted what it had learned.

Here is the difficulty. Both of these end up at roughly the same accuracy, near the majority-class floor. If you only look at the final accuracy number, the two are indistinguishable.

The only way to tell them apart is to look inside the model, and to look before training as well as after. That is exactly what our measurement design does.

4. Research question and testable hypotheses

1:00

So the primary question is this. How does depth change both predictive behavior and the geometry of the hidden representations?

We compare across two to thirty-two layers, holding the data and the training protocol fixed, so that depth is the only thing that moves.

We test three hypotheses. The first is that accuracy degrades unevenly across architectures. The second is that geometric collapse is not the same thing in every model. The third is that the mitigation which works depends on which failure mechanism is actually present.

These are stated in advance. They tell you what would count as a result before you see any of the numbers.

5. Benchmark and task specification

0:55

All experiments run on Cora. It is a citation network. Each node is a scientific paper, and each edge is a citation between two papers.

There are two thousand seven hundred and eight papers, five thousand two hundred and seventy-eight citation links, and seven research topics to predict. Each paper is described by one thousand four hundred and thirty-three binary word features.

We use the standard public split. One hundred and forty labeled nodes for training, five hundred for validation, and one thousand for testing.

One hundred and forty labels is a very small number. That is deliberate. It makes the quality of the learned representation matter.

6. Architectural controls isolate distinct propagation mechanisms

1:00

We compare four architectures, chosen because they aggregate information in genuinely different ways.

GCN uses symmetric normalized propagation. It is our reference point for depth-induced contraction.

GraphSAGE keeps the node's own features in a separate term from the neighborhood average, so self information and neighbor information are handled separately.

GAT learns attention weights over neighbors. That lets us ask whether adaptive weighting delays collapse.

GCNII is different in kind. It reintroduces the initial representation at every layer and adds an identity mapping. It was designed specifically to make depth work. So GCNII is not simply a fourth baseline. It is a control that tells us what happens when information is deliberately preserved.

7. Experimental protocol and reproducibility controls

1:05

The experimental design has three parts.

The first is the depth sweep. Four architectures, five depths from two to thirty-two, and ten independent random seeds for every configuration. Every number I report is a mean over ten seeds.

The second is the diagnostics. Alongside test accuracy we record two representation measures, mean average distance and Dirichlet energy. We record them at three points in every run: at initialization before any gradient step, at the checkpoint we actually report, and at the final epoch.

The third is the interventions. We test residual connections, PairNorm, and Jumping Knowledge on top of GCN, and we treat GCNII as the architectural control.

In total this is five hundred and thirty-four training runs, and every figure in the report regenerates from the stored records.

8. Representation diagnostics: formal definitions

1:15

These are the two instruments. The important thing is that they do not measure the same property.

Mean average distance, which I will call MAD, is built from cosine distance between node representations. It asks whether nodes point in the same direction. It is scale invariant. If you multiply every node representation by a positive number, MAD does not change.

Dirichlet energy asks something different. It measures how much representations differ across the edges of the graph, with degree normalization. It is quadratic in magnitude. If you scale the representation by c, the energy scales by c squared.

So one metric sees direction and ignores size. The other sees both.

That difference is deliberate. It means that when the two metrics disagree, the disagreement is informative rather than contradictory. Later in the talk I will show you a case where they disagree completely, and where that disagreement is the finding.

9. Result I — Depth degradation is architecture-dependent

1:15

Here is the first result.

All three conventional architectures peak at depth two. By depth four they have already lost between four and eight accuracy points. Beyond that they fall steeply.

At depth thirty-two, all three sit below thirty-one point nine percent. That number is the majority-class floor. It is the accuracy you get by ignoring the input entirely and always predicting the most common class. So these deep models are worse than a rule that does not look at the data at all.

Macro-F1 is worse still. GCN reaches zero point two one, GraphSAGE zero point zero five, GAT zero point zero five. That tells us the predictions concentrate onto a few classes rather than spreading errors evenly.

And then there is GCNII, on the right: eighty-three point three percent at thirty-two layers. That is its best result, at the greatest depth we tested. So depth is not uniformly harmful. It depends on the architecture.

10. Result II — Accuracy loss and geometric collapse diverge

1:05

Now the same runs, viewed through geometry rather than accuracy.

This is MAD against depth. GraphSAGE and GAT fall to essentially zero. Their node representations end up pointing in the same direction. That is directional collapse.

GCN does not. It holds around zero point one eight even at thirty-two layers.

But look back at the previous slide. All three lost roughly the same amount of accuracy.

So we have architectures with similar accuracy and completely different internal states. Accuracy is not telling us what happened inside the model. This is the point where a study that reported accuracy alone would have to stop.

11. Result III — GraphSAGE and GAT do not train at depth 32

1:30

This is the central result of the project. At depth thirty-two, GraphSAGE and GAT do not train at all.

The number on the left is the natural logarithm of seven, about one point nine five. That is the cross-entropy loss of a model that outputs a uniform guess over seven classes. It is the loss of knowing nothing.

GraphSAGE bottoms out at one point nine three. GAT at one point nine four. That is a movement of one or two hundredths of a nat. In practical terms, the loss does not move.

The second signal is early stopping. We allow one hundred epochs of patience. GraphSAGE and GAT exhaust it almost immediately, between one hundred and one hundred and sixty-eight epochs. GCN, by comparison, runs for three hundred to nearly nine hundred.

The third signal is the representation itself. Their MAD at the checkpoint is the same as their MAD before training started.

Three measurements, none of them computed from the others, all saying the same thing. The weights never moved. So calling this oversmoothing would be wrong. Nothing was learned, so there was nothing to oversmooth.

12. Result IV — Collapse is present before optimization

1:10

Let me now separate what the architecture does from what training does.

The epoch-zero capture is taken before any gradient step. The weights are random. So anything we see here is caused by the propagation operator and the initialization, not by learning.

At epoch zero, GCN and GAT lose both direction and magnitude as depth increases. Their energy ratio falls by about twelve and ten orders of magnitude by depth thirty-two.

GraphSAGE behaves differently. Its MAD collapses, but its energy does not vanish. It settles around nineteen.

That is a strange combination, and it is the subject of the next slide. The checkpoint and final captures then tell us what training changed.

13. Result V — Why GraphSAGE's two metrics disagree

1:20

So why do GraphSAGE's two metrics disagree?

We ran a singular value decomposition on its representations at depth sixteen, across three seeds.

The result is that the representation is exactly rank one. The top singular value carries the entire energy, a fraction of one point zero zero zero zero. Every node lies on a single line.

The question is which line. Its top singular vector matches the constant direction with cosine one point zero zero zero zero. Against the square root of degree direction it is zero point nine five.

Here is why that matters. Dirichlet energy is zero only on the null space of the graph Laplacian. On Cora, node degrees range from two to one hundred and sixty-nine. Because the graph is irregular, that null space is the square root of degree direction, not the constant vector.

So GraphSAGE collapses onto a direction where energy is not zero. Its representations align completely, MAD goes to zero, and the energy stays high. Both metrics are correct. They are answering different questions.

14. Result VI — Deep GCN contracts, then amplifies

1:10

GCN at depth thirty-two does something different again.

If we look at energy layer by layer at the checkpoint, the early layers have essentially collapsed. Their energy sits below our numerical floor of ten to the minus twelve, typically for the first twenty-two to twenty-eight of the thirty-one measured layers.

Then, near the output, the energy grows by many orders of magnitude. So the model contracts and then amplifies. Across the whole band the energy ratio spans twenty-six orders of magnitude between seeds.

Nine of the ten seeds show this pattern, so it is reproducible rather than one unlucky run.

I want to be careful here. We can see the pattern clearly. We did not record per-layer gradients, so we cannot tell you the parameter-level cause. That is recorded as an open

question.

15. Intervention study — Jumping Knowledge gives the largest recovery

1:05

Given all of that, what actually helps?

We tested three mitigations on GCN at depth thirty-two, against an unmitigated baseline of twenty-four percent.

Jumping Knowledge is the clear winner at seventy-three percent. It changes the readout rather than the propagation, so the classifier can draw on earlier layers instead of only the deepest one.

PairNorm reaches fifty-eight percent. It works directly on the geometry, rescaling representations so their spread does not shrink with depth.

Residual connections alone reach nineteen percent, which is below the baseline. Combining PairNorm with residual is also worse than PairNorm by itself.

So these are not interchangeable tools. They target different mechanisms, and combining them does not simply add their benefits.

16. Intervention transfer — the winning mitigation is architecture-dependent

1:15

Jumping Knowledge won on GCN, so we carried it to the other two architectures. Same mitigation, same protocol, ten seeds each.

On GAT it works. Seventy-two percent, essentially matching GCN's seventy-three. The macro-F1 values are almost identical as well.

On GraphSAGE it does not. It reaches forty percent. That is a real improvement, and all ten seeds sit above the majority-class floor, so I do not want to dismiss it. But it is about a third of what the same mitigation achieves on the other two.

The macro-F1 tells you more. Zero point two six, against zero point seven two for the other two. So even the accuracy it does recover is not spread evenly across the seven classes.

The conclusion is that you cannot choose a mitigation from the depth alone. It depends on which architecture you are trying to fix.

17. Mechanistic interpretation — GCNII preserves the initial signal

1:05

That brings me back to GCNII, the one architecture that improved with depth.

GCNII does two things. It reintroduces the original input representation at every layer, and it adds an identity mapping that limits how much each layer can transform its input.

The outcome is eighty-three point three percent at thirty-two layers. That is its best result, at the greatest depth tested, and with the tightest spread across seeds of any depth.

Its contraction slope also stays flat, within roughly plus or minus zero point four at every depth, where GCN's runs from about plus one point six to plus two point five.

This is consistent with the idea that preserving information across layers is what makes depth usable. I want to say clearly that it is consistent with, and not proof of. We did not inspect GCNII's own gradients.

18. CONCLUSIONS · SCOPE · NEXT STEPS

1:05

Three conclusions.

First, depth-related failure is architecture-specific. Two of our architectures never trained at depth thirty-two. One trained and then developed a contract-then-amplify signature. Treating those as the same phenomenon would be a mistake.

Second, accuracy and representation geometry have to be read together, and measured before training as well as after. That is what allowed us to separate a model that never learned from one that learned and then collapsed.

Third, the effective mitigation depends on the architecture. Jumping Knowledge transferred to GAT and not to GraphSAGE.

On scope, this is one dataset, one split, and ten seeds. We do not claim a universal depth limit. What we think transfers is the method, not the numbers.

Thank you. I am happy to take questions.